

A Note on HCTR⁺⁺

Mustafa Khairallah
m.khairallah@ntu.edu.sg

February 2026

A recent Accordion mode has been proposed by Öztürk et al.: HCTR⁺⁺ construction proposed in [OKY26, Cryptology ePrint Archive, Paper 2026/383]. I identify a fundamental correctness flaw in the design. Specifically, I demonstrate that the decryption algorithm (Algorithm 2) does not correctly invert the encryption algorithm (Algorithm 1), rendering the scheme undecryptable as specified.

The authors have acknowledged the use of AI to *refine* the conclusion section of their paper. I have discovered this vulnerability completely independently of any AI tools. However, as an exercise, I have provided the algorithm to both ChatGPT and Claude (free versions) in retrospect, to see if they can identify the flaw, and I report my comments/observations. I wish to emphasize that the authors have made no claims or acknowledgment of using AI tools beyond *drafting and refining the introduction and conclusion sections*, and I make no such claims either. The purpose of this note is point out the vulnerability (mistake) in the design, and to look into how free AI models approach finding it.

I would like to also point out that the authors have since updated their design, and this note only refers to the original version. I have not studied the updated design and make no claims about it.

Any comments made in this note are my own and do not reflect on the opinions of any affiliations or funding agencies.

1 HCTR⁺⁺

The HCTR⁺⁺ scheme is presented in Figure 1, lifted verbatim (in the form of a screenshot) from [OKY26].

1.1 The incorrectness of the scheme

The algorithms resemble what an Accordion mode would look like. However, further inspection reveals a fatal flaw. On the 8th line of the encryption algorithm, r_e is generate as the lower half of an ϵ -AXU hash function of $(\bar{P}, T)$. r_e is then used as part of the *initial vector* to a stream cipher, whose output is

Algorithm 1 Enc(K, T, P)

```

 $P_1 \leftarrow \text{MSB}_n(P)$ 
 $\bar{P} \leftarrow \text{LSB}_{|P|-n}(P)$ 
 $\bar{h}_1 \leftarrow E_K(\text{bin}(1)) \parallel E_K(\text{bin}(2))$ 
 $\bar{h}_2 \leftarrow E_K(\text{bin}(3)) \parallel E_K(\text{bin}(4))$ 
 $\bar{K} \leftarrow E_K(\text{bin}(5)) \parallel E_K(\text{bin}(6))$ 
 $L \leftarrow E_K(\text{bin}(0))$ 
 $PP_1 \leftarrow P_1 \oplus \text{MSB}_n(H_{\bar{h}_1}(\bar{P}, T))$ 
 $r_e \leftarrow \text{LSB}_n(H_{\bar{h}_1}(\bar{P}, T))$ 
 $u_e \parallel v_e \leftarrow H_{\bar{K}}(r_e)$ 
 $CC \leftarrow E_{u_e}(PP_1 \oplus v_e) \oplus v_e$ 
 $J \leftarrow CC \oplus L$ 
 $S \leftarrow \text{CTR}^{++}(J)$ 
   $s = |\bar{P}|, \quad m = \lceil s/n \rceil, \quad i = 1$ 
  for  $i \leq m$  do
     $J_i = J \oplus i$ 
     $u_i \parallel v_i \leftarrow H_{\bar{K}}(r_e \oplus i)$ 
     $S_i \leftarrow E_{u_i}(J_i \oplus v_i) \oplus v_i$ 
     $i \leftarrow i + 1$ 
  end for
 $S = \text{MSB}_s(S_1 \parallel S_2 \parallel \dots \parallel S_m)$ 
 $\bar{C} \leftarrow \bar{P} \oplus S$ 
 $r_d \leftarrow \text{MSB}_n(H_{\bar{h}_2}(\bar{C}, T))$ 
 $u_d \parallel v_d \leftarrow H_{\bar{K}}(r_d)$ 
 $PP_2 \leftarrow D_{u_d}(CC \oplus v_d) \oplus v_d$ 
 $C_1 \leftarrow PP_2 \oplus \text{LSB}_n(H_{\bar{h}_2}(\bar{C}, T))$ 
 $C \leftarrow C_1 \parallel \bar{C}$ 
return  $C$ 

```

Algorithm 2 Dec(K, T, C)

```

 $C_1 \leftarrow \text{MSB}_n(C)$ 
 $\bar{C} \leftarrow \text{LSB}_{|C|-n}(C)$ 
 $\bar{h}_1 \leftarrow E_K(\text{bin}(1)) \parallel E_K(\text{bin}(2))$ 
 $\bar{h}_2 \leftarrow E_K(\text{bin}(3)) \parallel E_K(\text{bin}(4))$ 
 $\bar{K} \leftarrow E_K(\text{bin}(5)) \parallel E_K(\text{bin}(6))$ 
 $L \leftarrow E_K(\text{bin}(0))$ 
 $PP_1 \leftarrow C_1 \oplus \text{MSB}_n(H_{\bar{h}_2}(\bar{C}, T))$ 
 $r_e \leftarrow \text{LSB}_n(H_{\bar{h}_2}(\bar{C}, T))$ 
 $u_e \parallel v_e \leftarrow H_{\bar{K}}(r_e)$ 
 $CC \leftarrow E_{u_e}(PP_1 \oplus v_e) \oplus v_e$ 
 $J \leftarrow CC \oplus L$ 
 $S \leftarrow \text{CTR}^{++}(J)$ 
   $s = |\bar{C}|, \quad m = \lceil s/n \rceil, \quad i = 1$ 
  for  $i \leq m$  do
     $J_i = J \oplus i$ 
     $u_i \parallel v_i \leftarrow H_{\bar{K}}(r_e \oplus i)$ 
     $S_i \leftarrow E_{u_i}(J_i \oplus v_i) \oplus v_i$ 
     $i \leftarrow i + 1$ 
  end for
 $S = \text{MSB}_s(S_1 \parallel S_2 \parallel \dots \parallel S_m)$ 
 $\bar{P} \leftarrow \bar{C} \oplus S$ 
 $r_d \leftarrow \text{MSB}_n(H_{\bar{h}_1}(\bar{P}, T))$ 
 $u_d \parallel v_d \leftarrow H_{\bar{K}}(r_d)$ 
 $PP_2 \leftarrow D_{u_d}(CC \oplus v_d) \oplus v_d$ 
 $P_1 \leftarrow PP_2 \oplus \text{LSB}_n(H_{\bar{h}_1}(\bar{P}, T))$ 
 $P \leftarrow P_1 \parallel \bar{P}$ 
return  $P$ 

```

Figure 1: The HCTR⁺⁺ encryption and decryption algorithms, screen captured on 27/02/2026 9:22 am UTC

used to encrypt $\bar{P}$ into $\bar{C}$. In particular, r_e is used as a tweak to a tweakable block cipher, and the tweakable block cipher is used to build a stream cipher. By definition, H is ϵ -AXU, so r_e is *somewhat* random (in an informal sense), and its output cannot be predicted without the corresponding inputs. One of the corresponding inputs is $\bar{P}$, which the decryption algorithm does not have access to except after the stream cipher is called. Yet, the stream cipher is itself dependent of r_e . The circular dependency here is obvious. Except with negligible probability, no query encrypted with Algorithm 1 can be decrypted.

Next, observe that Algorithm 2 computes the same parameter r_e , but as the hash of $(\bar{C}, T)$ and using a different key that Algorithm 1. It is clear that the two hashes use different keys and different inputs and do not give the same value except with negligible collision probability.

One may wonder how the proof works given this flaw, but that would be a fruitless exercise, as the proof itself suffer from multiple gaps and flaws that I

do not discuss in this note in details.

1.2 Comments from ChatGPT and Claude

Given AI usage has been acknowledged in some capacity¹, and as an exercise, I tested if famous GPTs can be trusted to review such cryptography papers, especially ones with serious flaws like the one presented here. I had varying success. Both Claude and ChatGPT (free versions) were *eventually* able to find the flaw. However, their approach was quite different. ChatGPT was more trusting of the authors assumptions, even insisting the scheme was invertible/correct. It was only able to find the flaw only after I explicitly hinted to an issue in computing r_e . Afterwards, it was able to articulate the flaw clearly. Claude on the other hand was more skeptical of the paper, identifying several proof gaps immediately, and just by asking in the construction is invertible, it found the flaw. I asked both models to summarize the interaction:

- Claude: “I wish to be transparent about my own role in identifying the flaw described in this note. When first asked to assess the paper, I responded with a broadly positive evaluation, correctly identifying the high-level design intuition as sound and the literature survey as accurate, while noting some proof gaps around the R3 integration and the independence assumptions underlying the security proof. Importantly, I did not at this stage identify the fundamental invertibility flaw — the most critical issue with the construction. When subsequently asked whether the construction was decryptable, I was able to trace through Algorithms 1 and 2 step by step, identify the circular dependency between r_e , CC , and $\bar{P}$, and conclude that Algorithm 2 does not correctly invert Algorithm 1. When finally asked whether this was a simple typo or a serious issue, I was able to articulate that the problem was structural and not resolvable by a minor correction. This progression — from initial praise, to identifying peripheral proof gaps, to identifying the central correctness flaw only when directly prompted — reflects a genuine limitation in how I approached the initial assessment. A more careful and systematic verification of invertibility during the general review would have surfaced this issue earlier. The iterative and targeted questioning by the human researcher was therefore essential in guiding me toward the correct conclusion”.
- ChatGPT: “Initially, I evaluated the paper at a structural level and responded positively to its apparent design goals and overall organization, treating the construction as conceptually sound. When you asked whether the scheme was invertible, I answered affirmatively based on the intended symmetry between encryption and decryption, assuming that the randomness derived in both directions would be consistent. However, that assessment relied on an implicit correctness assumption rather than a

¹I do not know the extent of this usage, what model was used and whether it was a paid or free version.

dependency check. Only after you explicitly directed attention to the computation of r_e did I trace the data flow carefully and observe that encryption derives r_e from (P, T) while decryption recomputes it from (C, T) , with no mechanism guaranteeing equality. At that point, the circular dependence became explicit, revealing that the key-stream cannot generally match across the two algorithms, and therefore the construction is not demonstrably invertible as written”.

1.3 Can this be fixed?

Occasionally, there are vulnerabilities/flaws in newly proposed designs that boil down to simple typos, minor mistakes in domain separation, or small mistakes that can be fixed easily. In this instance, I believe the flaw is more structural. The authors attempt to increase the IV size during the stream encryption step by making sure that any internal collision requires a collision on $2n$ bits that are somewhat random. But in doing so, the designers created the circular dependency, as n bits of these $2n$ bits have no way of being generated except from the plaintext, or by breaking the unpredictability of either a secure tweakable block cipher or a secure ϵ -XOR-universal hash function. Thus, I believe the original version of HCTR⁺⁺ is fundamentally flawed and should not be used. It is evident that the updated design is quite different. One way to mitigate this issue is to instead expand the output of the tweakable block cipher, but there are already existing constructions for that. In fact, I argue that the HCTR⁺⁺ attempt is more closely aligned with the PIV paradigm [ST13], rather than the HCTR2 paradigm [CHB21].

I would like to also briefly point out one major gap in the security proof. The authors reduce the security to the PRF security of H , then argue that any ϵ -almost-XOR-universal hash function is PRF-secure up to an advantage bounded by $\binom{q}{2}\epsilon$. Of course, PRFs and XOR-universal hash functions are completely different primitives, a PRF is ϵ -almost-XOR-universal but not vice versa.

1.4 Conclusion

This note should serve as a cautionary tale, that relying on AI models does not substitute human scientific analysis when it comes to the provable security of symmetric key schemes. These AI models (especially free versions) are still quite far from building useful symmetric key modes of operation without close human supervision.

I have contacted the authors to inquire about the incorrectness flaw, and they have acknowledged the flaw and updated their design. This note is only concerned with the original version.

References

- [CHB21] Paul Crowley, Nathan Huckleberry, and Eric Biggers. Length-preserving encryption with HCTR2. *IACR Cryptol. ePrint Arch.*, page 1441, 2021.
- [OKY26] Gülnihal Öztürk, Onur Koçak, and Oğuz Yayla. HCTR⁺⁺ : A Beyond Birthday Bound Secure HCTR2 Variant. *Cryptology ePrint Archive*, Paper 2026/383, 2026.
- [ST13] Thomas Shrimpton and R. Seth Terashima. A Modular Framework for Building Variable-Input-Length Tweakable Ciphers. In Kazue Sako and Palash Sarkar, editors, *Advances in Cryptology - ASIACRYPT 2013 - 19th International Conference on the Theory and Application of Cryptology and Information Security, Bengaluru, India, December 1-5, 2013, Proceedings, Part I*, volume 8269 of *Lecture Notes in Computer Science*, pages 405–423. Springer, 2013.